



Model Deployment Fundamentals

LECTURE

Model Management

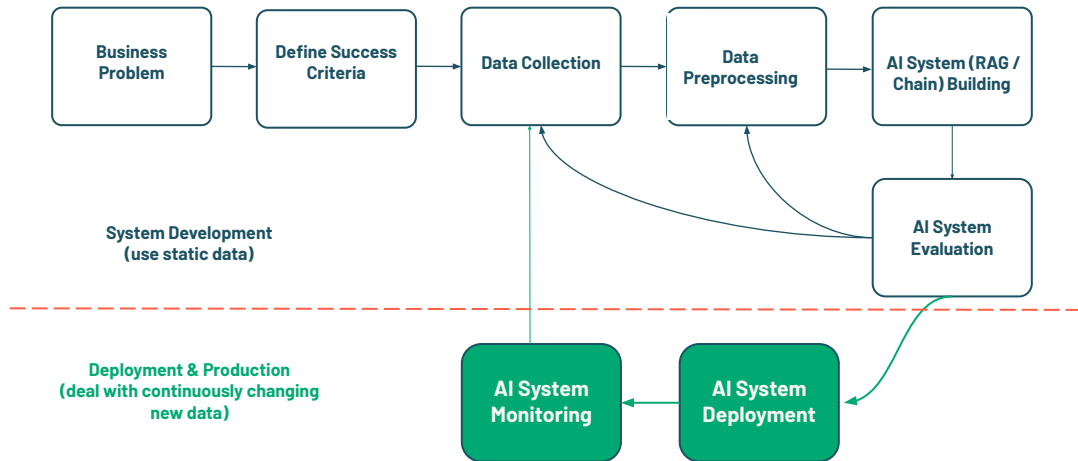


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

In this lecture Model Management, we cover the Gen AI system lifecycle, GenAI deployment strategies, MLflow components and model flavors, integrating MLflow with the development lifecycle, Unity Catalog features, using MLflow's Unity Catalog Model Registry, and how MLflow facilitates GenAI deployment.

The Gen AI System Lifecycle

Your AI System is ready, then what?



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Managing a machine learning or generative AI project involves several key steps. First, start with a well-defined business problem and set clear success criteria using existing processes or baseline models as a comparison. Next, gather or prepare quality data for training or evaluation, which might involve technical writers or pre-processing the data to make it usable for machine learning or LLM systems.

After preparing the data, build the system—this could involve fine-tuning, pre-training, or assembling components like RAG chains. Begin with offline evaluation to test system performance, refining data preparation if results aren't satisfactory. Once the model passes offline checks, deploy it and monitor its behavior in production, always maintaining a feedback loop so new data can continually improve the model over time.

GenAI Deployment

Packaging models/pipelines

With LLMs, **ML logic is packaged in new forms** which could include:

- An engineered prompt (possibly stored as a template)
- A “chain” from LangChain, LlamaIndex etc.
- A lightweight call to an LLM API service such as:
 - (Internally/Self) hosted foundation models (e.g. DBRX)
 - External proprietary models providers (e.g. OpenAI)
- A lightweight call to a bespoke (self) hosted LLM API:
 - Fine-tuned model
 - Pretrained model
- Locally invoking an LLM or an LLM+tokenizer pipeline (e.g. Hugging Face pipeline) running on GPU(s)

**All of these still
constitute
models and
pipelines!**



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

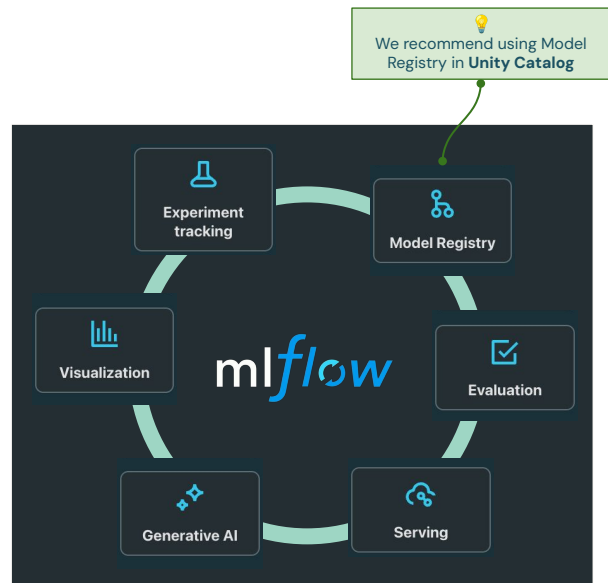
When packaging generative AI or language models, the logic can take several forms. It might be an engineering prompt—precise instructions given to the model to guide output. The system could also use a chain-based approach, like LangChain or LlamaIndex, or something custom-coded in Python to combine different components.

Often, models are accessed using API calls, which could point to internal or self-hosted foundation models, open source models, or external providers like OpenAI. These APIs can also serve bespoke, fine-tuned, or custom-trained models. Alternatively, models can run locally on GPUs or CPUs, especially for batch processing, where the model and pipeline are fully loaded onto the hardware. All of these approaches—prompts, chains, APIs, and local pipelines—define how a generative AI model or chain is packaged and deployed.

MLflow Components

Model management with MLflow

- Manage end-to-end ML and GenAI workflows, from development to production.
- Unified platform for both traditional ML and GenAI applications.
- Generative AI specific model flavors and evaluation metrics.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

To package generative AI models efficiently, MLflow plays an important role. MLflow is an open-source tool, not limited to Databricks, that helps manage the entire machine learning and generative AI workflow—from development through to production. When paired with Unity Catalog, it offers a comprehensive experience for tracking all data and AI assets in one place. MLflow also supports generative AI-specific model types and evaluation metrics, making it easier to monitor and assess these advanced models throughout their lifecycle. To package generative AI systems cleanly, MLflow is a key solution. MLflow is an open-source tool that is not exclusive to Databricks—it manages the entire machine learning and generative AI workflow from development to production. Combined with Unity Catalog, it lets you organize and track all your data and AI assets in one place. MLflow now supports generative AI models and includes evaluation metrics tailored for them, which will be discussed in more detail later in the lesson.

MLflow – Model (“Flavor”)

A standard format for packaging machine learning models

- Each **MLflow Model** is a directory containing arbitrary files, together with an `MLModel` file.
- `MLModel` file can define **multiple flavors** that the model can be viewed in.
- With MLflow Models deployment tools can understand the model.
- Model file can contain **additional metadata** such as signature, input example etc.

```
mlflow LangChain Flavor

# Directory written by
mlflow.langchain.log_model(model, "chain",...)
chain/
├── model
│   ├── steps
│   └── steps.yaml
├── MLModel
├── lc_model.py
├── conda.yaml
├── python_env.yaml
└── requirements.txt

# MLModel file
artifact_path: chain
flavors:
  langchain:
    code: null
    langchain_version: 0.1.5
    model_data: model
  python_function:
    loader_module: mlflow.langchain
```



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Since language model applications can be complex, MLflow now offers new model flavors designed for generative AI. A “flavor” in MLflow adds extra details—like metadata, datasets, metrics, and other artifacts—on top of the original model to help track its lifecycle. For example, when using LangChain, MLflow can log the entire chain as a model flavor, recording all steps, dependencies, environment settings, and runtime versions. This makes it easy to package, reproduce, and manage even complex logic as a unified MLflow model. Because generative AI logic can be complex, MLflow has introduced new model flavors to handle these cases. A flavor in MLflow adds useful metadata, datasets, metrics, input examples, and other artifacts on top of your base model for easier lifecycle tracking. For example, if you log a chain built with LangChain, MLflow tracks all steps, dependencies, environment info, and even Databricks runtimes—packaging everything needed to reproduce and manage the complex logic as a single MLflow model.

MLflow – Model (“Flavor”)

Built-in model flavors

Python Function (`mlflow.pyfunc`):

- Serves as a **default model interface** for MLflow Python models.
- Any MLflow Python model is expected to be loadable as a python function.
- Allows you to deploy models as Python functions.
- It includes all the information necessary to **load and use a model**.
- Some functions: `log_model`, `save_model`, `load_model`, `predict`

mlflow Model Flavors

Example built-in model flavors:

- LangChain
- OpenAI
- HuggingFace Transformers
- PyTorch
- TensorFlow
- ONNX
- **Python function**
- ...

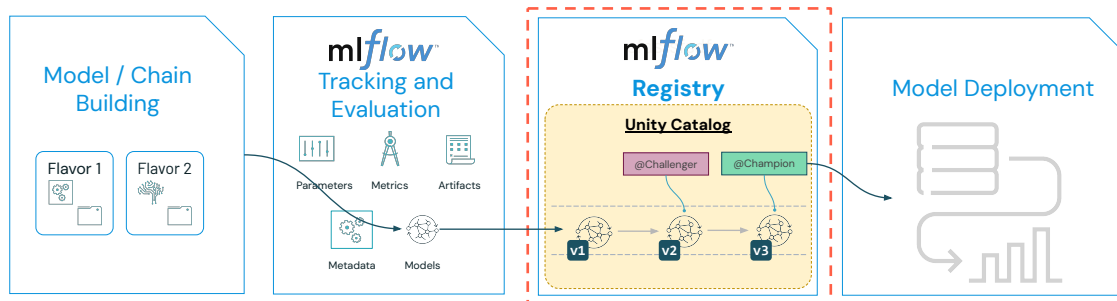


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

If your application logic isn't built using a pre-supported flavor—like LangChain, OpenAI, or Hugging Face transformers—you can use MLflow's pyfunc flavor. This provides a default model interface with a predict method, letting you wrap any custom Python code or API calls inside it. The pyfunc flavor is versatile for custom chains and production systems, allowing you to track and manage models even if they don't use standard frameworks.

MLflow and Development Lifecycle

MLflow integration to seamlessly deploy models



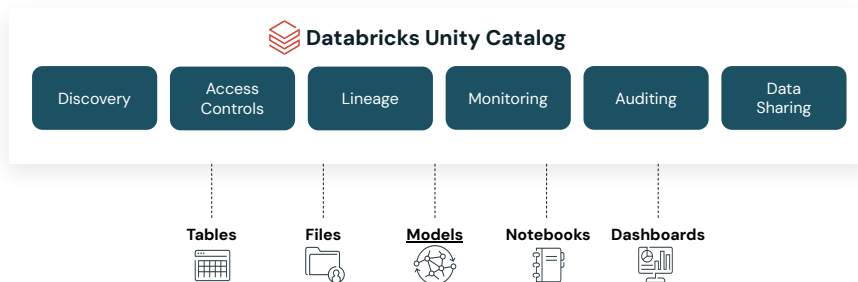
© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Integrating model flavors with MLflow streamlines the entire lifecycle of machine learning or language models. You start by creating different versions of your model or chain using a suitable flavor, then experiment and use MLflow to automatically track metadata, metrics, and parameters. Once a version performs well in offline evaluation, you push it to the MLflow registry, which manages DevOps or LLMOps tasks like versioning and deployment. Models can be tagged or given aliases in the registry, making it easier for deployment pipelines to select the right version for production. The Unity Catalog model registry further enhances this by offering robust organization and tracking of registered models.

Unity Catalog (UC)

Single governance solution for **data and AI assets** on the Lakehouse:

- Unified visibility into data and AI
- Single permission model for data and AI
- Open data sharing



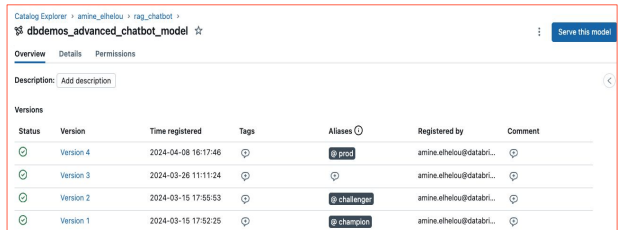
© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Unity Catalog provides a single governance solution for both data and AI assets, enabling end-to-end lineage between them. It uses a unified permission model and an open data sharing protocol, making it easy to share models with third parties or outside organizations. Managing models in Unity Catalog means assets are organized, secure, and can be shared flexibly while maintaining traceability and control throughout their lifecycle.

MLflow – Unity Catalog Model Registry

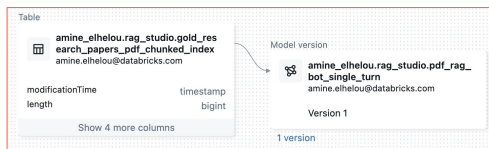
A centralized model store

- **Model** lifecycle management with **versioning** & **@aliases**. (e.g. @champion/challenger)
- Deploy and organize models
- Collaboration and ACLs
- Full model lineage
- Tagging and annotations



The screenshot shows the 'dbdemos_advanced_chatbot_model' page in the Unity Catalog Model Registry. It features a table with columns for Status, Version, Time registered, Tags, Aliases, Registered by, and Comment. The table lists four versions of the model, with Version 4 being the latest and Version 1 being the initial version. The Aliases column shows '@prod' for Version 4, '@challenger' for Version 2, and '@champion' for Version 1.

Status	Version	Time registered	Tags	Aliases	Registered by	Comment
🟢	Version 4	2024-04-08 16:17:46	🔍	@prod	amine.ehelou@databri...	🔍
🟢	Version 3	2024-03-26 11:11:24	🔍	🔍	amine.ehelou@databri...	🔍
🟢	Version 2	2024-03-15 17:55:53	🔍	@challenger	amine.ehelou@databri...	🔍
🟢	Version 1	2024-03-15 17:52:25	🔍	@champion	amine.ehelou@databri...	🔍



The screenshot shows a 'Table' view of the model registry. It displays a table with columns for modificationTime, length, timestamp, and bigint. The table lists the model 'amine_ehelou_rag_studio_gold_res' and its version 'amine_ehelou_rag_studio_pdf_rag_bot_single_turn'. The version is labeled 'Version 1' and '1 version'.

modificationTime	length	timestamp	bigint
amine_ehelou_rag_studio_gold_res	amine_ehelou_rag_studio_pdf_rag_bot_single_turn	amine.ehelou@databricks.com	Version 1



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

At a high level, Unity Catalog manages AI models through automatic versioning and tagging based on product names, business keywords, or project aliases. Aliases help track models across development stages and make it easy to select the correct model for production. You can control who can access, manage, create versions, and promote models—internally or externally—while also maintaining visibility into their lineage throughout the workflow.

MLflow and GenAI Deployment

Benefits of MLflow

Dependency & Environment Management

- Ensures that the deployment environments matches the training environment.
- Ensures that models are run consistently, regardless of where they are deployed.

Packaging Models and Code

- Any code and configuration are packaged.
- Ensures model can be deployed seamlessly without any missing components

Multiple Deployment Options

- Built-in local Flask Server with MLServer
- Deploy to major cloud providers.
- **Deploy with Databricks Model Serving**



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

MLflow helps manage machine learning models by automatically tracking which package versions and dependencies were used to train your model. This means you don't have to manually set up environments when moving models between testing and production—they'll run with the right software automatically. With model flavors, MLflow lets you bundle your code, configuration files, and artifacts into a single package. This makes it easy to deploy your model for local testing and seamlessly scale up to production, especially with Databricks' one-click deployment options. This way, model transitions between environments are reliable, fast, and simple.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Thank you for completing this lesson and continuing your journey to develop your skills with us.